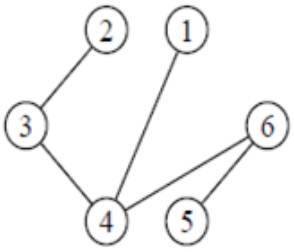
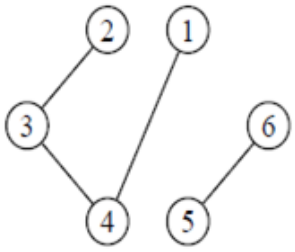
Notions de base des arbres

1- Définition :

On appelle **arbre** tout graphe connexe sans cycle, et on appelle **forêt** tout graphe non connexe et sans cycle. Une **feuille** ou **sommet pendant** est un sommet de degré 1.



Arbre
Les sommets 1, 2 et 5 sont les feuilles
ou sommet pendant



Forêt
Les sommets 1, 2, 5 et 6 sont les feuilles
ou sommet pendant

2- *Théorème

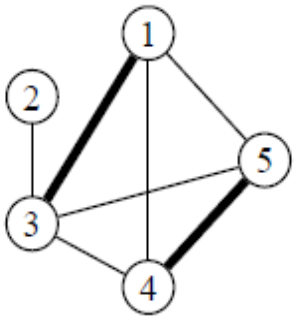
Les affirmations suivantes sont équivalentes pour tout graphe *G* à *n* sommets, *G* est un arbre :

1. *G* est connexe et sans cycle.
2. *G* est sans cycle et comporte *n*−1 arêtes.
3. *G* est connexe et comporte *n*−1 arêtes.
5. chaque paire (*u*, *v)* de sommets distincts est reliée par une seule chaîne simple (et le graphe est sans boucle).

3- Couplages

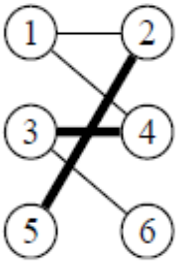
En théorie des graphes, un **couplage** ou **appariement** (en anglais *matching*) d'un graphe est un ensemble d'arêtes de ce graphe qui n'ont pas de sommets en commun, on peut aussi dire qu'un couplage (ou appariement) est un ensemble d'arêtes deux à deux non-adjacentes, un sommet ***v*** est **saturé** par un couplage ***C*** si ***v*** est l'extrémité d'une arête de ***C***, dans le cas contraire, ***v*** est **insaturé**. Un **couplage maximum** est un couplage contenant le plus grand

nombre possible d'arêtes, un graphe peut posséder plusieurs couplages maximum.

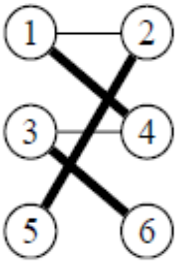


En gras, un couplage maximum de G. Les sommets 1, 3, 4 et 5 sont saturés.

Un **couplage parfait** est un couplage où chaque sommet du graphe est saturé.



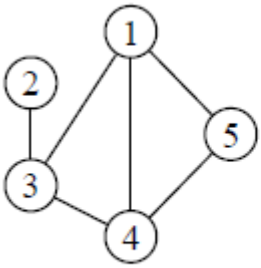
Un couplage



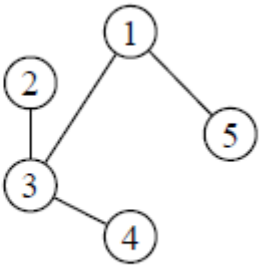
Un couplage maximum et parfait

4- Les arbres couvrant ou arbre maximal

Un **arbre couvrant** aussi appelé **arbre maximal** est un graphe partiel qui est aussi un arbre et qui connecte tous les sommets du graphe.



Graphe G



Un arbre couvrant

5- Arbre couvrant de poids minimum

Soit le graphe $G = (V,E)$ avec un poids associé à chacune de ses arêtes, on veut trouver dans le graphe G, un arbre maximal $A = (V,F)$ de poids total minimum.

5.1 - Algorithme de Kruskal (1956)

Données :

– Graphe $G = (V,E)$ ($|V| = n$, $|E| = m$)

– Pour chaque arête e de E , son poids $c(e)$.

Résultat : Arbre ou forêt maximale $A = (V,F)$ de poids minimum.

Trier et renuméroter les arêtes de G dans l'ordre croissant de leur poids :

$$c(e_1) \leq c(e_2) \leq \ldots \leq c(e_m).$$

Poser $F := \varnothing$, $k := 0$

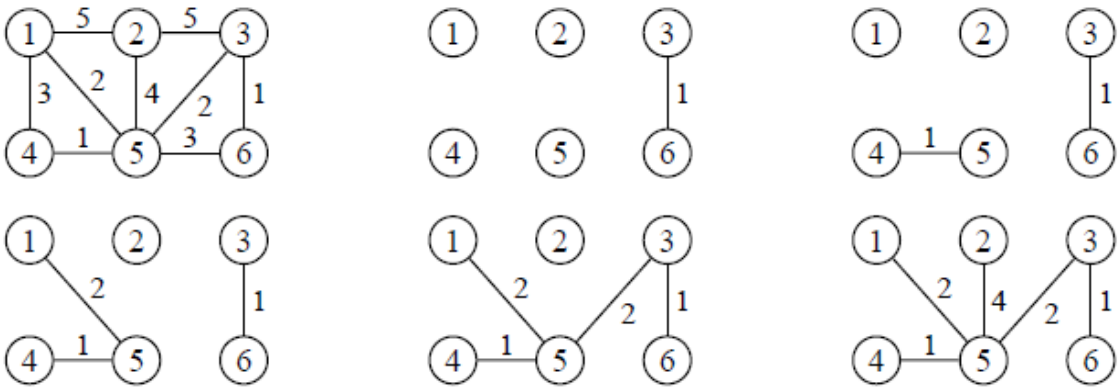
Tant que $k < m$ et $|F| < n-1$ faire

Début

si $ek+1$ ne forme pas de cycle avec F alors $F := F \cup \{ek+1\}$ $k := k+1$

Fin

Exemple



Explication

Notre graphe est connexe sans cycle et pondéré, on cherche un arbre qui couvre le graphe de poids minimal qui passe par tout les sommets : arbre couvrant de poids minimum.

1/ trier les arêtes par poids croissant :
3,6 : 1
4,5 : 1
1,5 : 2
3,5 : 2
1,4 : 3
2,5 : 4
1,2 : 5
2,3 : 5
La construction de l'arbre se base sur cet ordre des arêtes, à chaque étape on regarde l'arête courante et on l'ajoute a l'arbre a condition qu'elle ne forme pas de cycle avec les arêtes ajoutée.
Les arêtes de poids 3 n'ont pas pu être placées, car elles auraient formé un cycle, l'algorithme s'est arrêté dès que cinq arêtes ont été placées. Toute arête supplémentaire aurait créé un cycle, s'il y a plusieurs arêtes de même poids, il peut y avoir plusieurs arbres couvrants de poids minimum : tout dépend de l'ordre dans lequel ces arêtes ont été triées.
Poids de cet arbre : poids (T) : 1+1+2+2+4 = 10.

5.2 - Algorithme de Prim

L'algorithme consiste à faire croître un arbre depuis un sommet. On part d'un sous-ensemble contenant un sommet unique, à chaque itération on agrandit ce sous-ensemble en prenant l'arête incidente à ce sous-ensemble de coût minimum, toute arête qui crée un cycle est refusé.

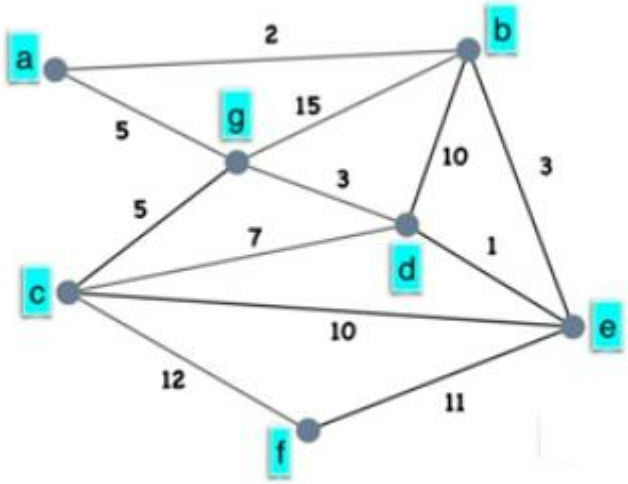
```
fonction prim(G, s)
       pour tout sommet t
              cout[t] := +∞
              pred[t] := null
       cout[s] := 0
       F := file de priorité contenant les sommets de G
avec cout[.] comme priorité
       tant que F ≠ vide
            t := F.defiler
            pour toute arête t--u avec u appartenant à
F
                si cout[u] >= poids de l'arête entre
les sommets t et u
```

```
                        pred[u] := t
                        cout[u] := poids de l'arête
entre les sommets t et u
                        F.notifierDiminution(u)

        retourner pred

        Fin
```

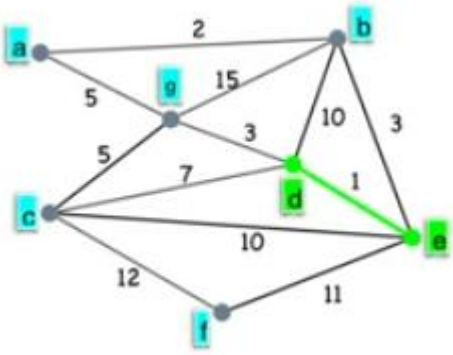
Exemple :



Explication :

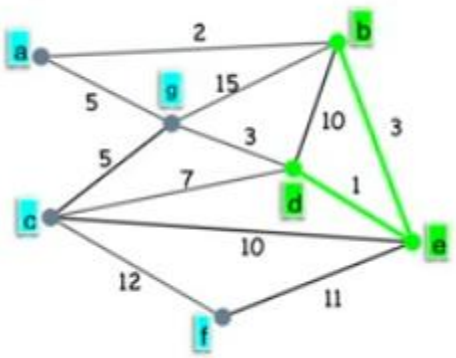
Notre graphe est connexe sans cycle et pondéré, on cherche un arbre qui couvre le graphe de poids minimal qui passe par tout les sommets : arbre couvrant de poids minimum.

Première étape :on part par exemple du sommet **d** sommet de départ qui peut être vu comme un arbre partiel auquel on va ajouter un sommet et une arête, l'arête qui va être choisi est celle qui sort de l'arbre et qui a un poids minimal, dans notre cas sa sera l'arête d,e = 1.



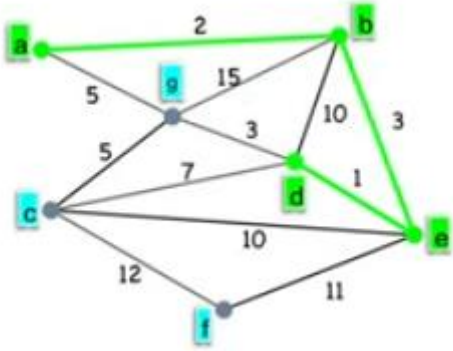
étape 1

A la deuxième étape : on regarde toutes les arêtes qui ont une extrémité dans l'arbre et hors de l'arbre, il faut que l'arête choisi soit de poids minimal et ne forme pas de cycle, on a le choix entre l'arête e,b = 3 et l'arête d,g = 3, on choisi l'arête e,b par exemple.

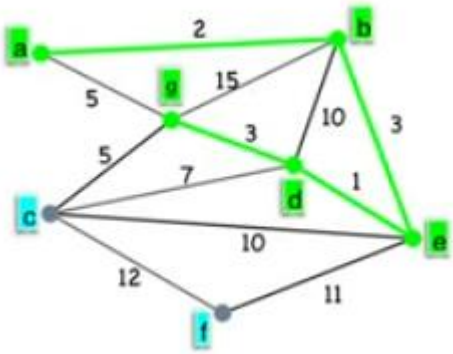


étape 2

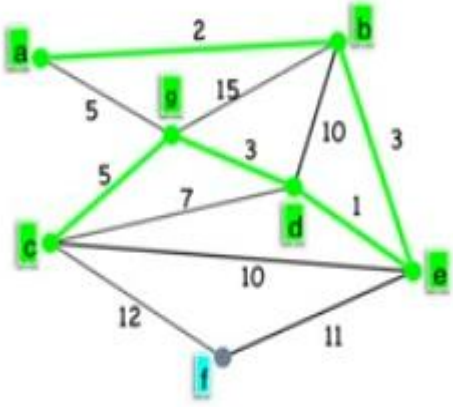
En suite en répétant l'algorithme avec le même principe on obtient :



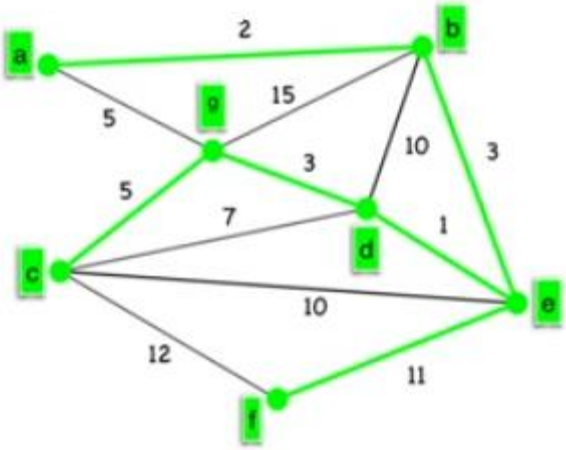
étape 3



étape 4



étape 5



étape 6

On a un arbre couvrant qui contient tous les sommets de poids minimal.

Poids de cet arbre : poids (T) = 2+3+1+3+5+11=25.